

Transformer Models in autonomous driving applications

Linus Jürgens

Abstract—Transformer architectures are gaining increasing relevance in the field of autonomous driving. Currently, most transformer models are applied in either the perception or the prediction stack, because they offer several advantages compared to classical approaches. This review paper focuses on the development of transformer architectures within these two components. Transformers are capable of capturing a global representation of a scene and can model relationships between observed entities. However, this also comes with several drawbacks, since they require large amounts of data to perform well and are currently not easily interpretable. This review demonstrates that classical approaches are increasingly being surpassed and that transformer models achieve new state-of-the-art results. This highlights the importance of further developing these models for autonomous driving applications, despite the significant challenges that remain.

I. INTRODUCTION

A large number of research groups worldwide are currently addressing on one of the most relevant research challenges: autonomous driving. An autonomous vehicle must perceive its environment in a precise and reliable manner, make predictions about possible future changes in the scenario, and react accordingly to ensure the safety of all involved participants. All of these processes must be executed in real-time. These challenges motivate the development of systems that operate robustly and reliably. The architecture of an autonomous driving system is typically divided into three components: perception, prediction, and control, as illustrated in Fig. 1. The perception module records sensor data relevant to the current state of the environment. The subsequent prediction module forecasts future movements and interactions of the detected objects. Finally, the control module uses this information to execute an appropriate maneuver. A key objective of many research efforts is to achieve an end-to-end (E2E) architecture, meaning that all components are learned within a single model. In contrast, the automotive industry still relies primarily on modular systems, which provide better interpretability and are therefore widely used in current frameworks. In recent years, transformer architectures have gained considerable importance because they can capture global context, process flexible input lengths, and integrate multimodal information. This makes them suitable for highly complex and dynamic situations such as autonomous driving. While transformers are rarely used in the control module, they have led to significant advances in both the perception and prediction stacks. Both stacks can be further divided into subcategories. In perception, object-based, grid-based, and sensor-fusion approaches exist. In prediction, trajectory-based and grid-based methods are commonly used. Due to space constraints, this review focuses on transformer models

within object-based perception and trajectory-based prediction. Other approaches are briefly mentioned for completeness.

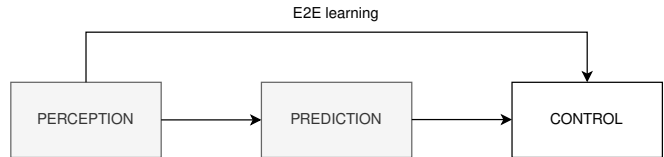


Fig. 1: Stacks for autonomous driving applications: Perception, Prediction, Control. They are connected with end-to-end learning (E2E).

II. TRANSFORMER ARCHITECTURE

A. Basic Transformer

The transformer architecture was introduced in the paper “Attention Is All You Need” in 2017 [1]. With this transformer architecture, the authors were able to outperform classical approaches such as CNNs or RNNs. This demonstrates the potential of transformer models when trained appropriately. Moreover, transformers are highly parallelizable, enabling more efficient training compared to sequential models. The key component of the architecture is the attention mechanism, which enables the model to capture relationships within the input sequence and thereby enabling the modeling of complex dependencies [1]. The input sequence is first tokenized into individual tokens. The attention mechanism is based on the matrices Q (query), K (key), and V (value). The attention function takes a query and a set of key-value pairs as input and computes the output as a weighted sum of the values. These weights reflect the relevance of each key with respect to a given query. Attention is calculated using the scaled dot-product, as illustrated in Fig. 2, which forms the foundation of multi-head attention and therefore of the entire transformer architecture. The scaled dot-product takes Q, K, and V as input and computes the attention, i.e., the relation between the tokens using the softmax function, as shown in Eq. 2. The scaling factor

$$\frac{1}{\sqrt{d_k}} \quad (1)$$

is used to prevent the softmax from becoming unstable when the dimensionality is large. This mechanism allows transformers to explicitly model relationships between tokens. The transformer architecture applies multiple such operations in parallel, known as multi-head attention. For this, Q, K, and V are linearly projected, and each head processes the input from its own perspective. Each head then applies the scaled dot-product attention.

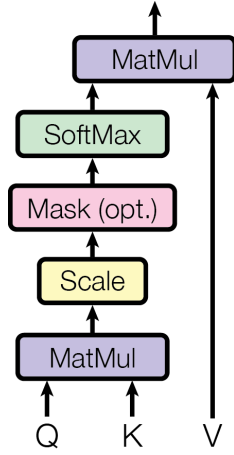


Fig. 2: This figure shows the attention mechanism with Q , K , and V matrices. Reproduced from [1].

Formally, the attention calculation is defined as:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (2)$$

After explaining the main components of the transformer, the focus shifts to the structure of the network. The transformer is based on an encoder decoder structure. The encoder builds the contextual representation z of the input x , and the decoder uses this representation to produce an output y . In addition, positional embeddings are incorporated to preserve information about the relative positions of tokens within the sequence.

B. Vision with Transformer

Transformer models were originally developed for natural language processing. However, the transformer concept quickly attracted attention in other fields such as computer vision, and in 2021 the Vision Transformer (ViT) was introduced for the first time [2]. This work applies the transformer architecture to image classification. The main result demonstrates that ViT can outperform classical CNNs when trained with sufficient data. This finding is also relevant in the context of autonomous driving, because it demonstrates that transformer architectures can be used in computer vision tasks. In the Vision Transformer, the architecture from the original paper [1] is not changed significantly. However, to use the transformer on images, the 2D image with dimensions $H \times W \times C$ must be transformed into a 1D sequence, because the transformer only accepts a 1D input. ViT additionally introduces a learnable class token, which was introduced in the BERT paper [3]. This CLS token represents the overall meaning of the input image after processing. The Vision Transformer is essential for autonomous driving research because, for the first time, it provides an alternative to CNN-based approaches and enables further investigations with a fully transformer-based architecture. With the publication of the ViT paper, it became possible for transformer models to analyze images directly, without using

a CNN backbone. As described in the paper, ViT outperforms previous state-of-the-art models. It is particularly effective at capturing global contextual information. However, ViT also has important limitations. One major limitation is that it requires very large amounts of data to perform well; when trained on smaller datasets, its performance drops significantly. In addition, ViT has very little inductive bias, which is problematic for autonomous driving scenarios because available training data is limited. This limits the practicality for autonomous driving applications and causes CNNs to outperform it unless massive pretraining has been used. Furthermore, the basic ViT is designed only for image classification. It does not produce bounding boxes, which makes it unsuitable for autonomous vehicles that need to precisely detect multiple objects in the environment [2].

III. TRANSFORMER IN AUTONOMOUS DRIVING APPLICATIONS

Autonomous driving is a multimodal problem, as it requires the joint processing of data from multiple heterogeneous sources. Furthermore, dynamic scenes arise due to the large number of traffic participants. As a result, the system must operate in real-time while maintaining awareness of the global scene context. Moreover, any error in an autonomous driving scenario can have severe consequences. For this reason, safety is the highest priority for such systems. The high level of complexity in these systems motivates the use of transformer models. Through multi-head self-attention, transformers are able to model and analyze complex relationships. Furthermore, they can process information from multiple sensors, enabling a more comprehensive scene understanding. In autonomous driving applications, two main tasks must be addressed. The first task is perception, which corresponds to the vehicle's visual understanding of its environment. In this step the vehicle must obtain an accurate representation of its surroundings with all objects and with sensors it has. The second task is the prediction, in this step the future motion of objects must be predicted to enable timely reactions. Therefore it needs complex systems, because there are no fixed trajectories, which can lead to problems during the driving. For these challenges, classic methods are insufficient, cause CNNs are limited locally and with that they are losing the global image context [2]. RNNs, on the other hand, struggle to model long-term temporal dependencies [1] and GNNs only represent local graph relation [4]. Transformers address these limitations, as it provides a global understanding, can describe time dependencies and is flexible with the input length. With recent advances in hardware, the relevance of transformers in autonomous driving has increased, as their computational demands require powerful hardware. Further there are huge datasets like NuScenes or Waymo which allows to train such models which needs a lot of data. There are still some difficulties, that is the reason why transformers are not completed yet. However, their real-time performance is still limited and the high computational cost makes deployment challenging in vehicles with constrained onboard resources.

The following section discusses how transformer models are used for object detection in autonomous driving systems.

A. Transformer in Perception

As discussed earlier, the Vision Transformer is not capable of detecting objects in autonomous driving scenarios and is therefore not suitable as a perception model in autonomous driving. At this stage, a model capable of detecting multiple objects is required, one of those is the DETection TRansformer, DETR [5]. This model is trained end-to-end using a dedicated loss function. This loss formulation enables bipartite matching between predictions and ground-truth objects. DETR differs from ViT, as for the feature extraction out of the original image it uses a CNN backbone and produces a feature map. This feature map is flattened into a one-dimensional representation and then it becomes the input for the transformer. The architecture of the transformer is a basic encoder-decoder structure. The input has no inherent information about spatial location in the original image, therefore it adds a fixed positional encoding. Then the input is going to the encoder, where with the help of self-attention the relationships between features are learned. Then we get the representation of the input which shows the relation between the features in the picture. The decoder then processes the encoded features using N learnable object queries and in this step the objects are identified, with the help of self attention. The detected objects are subsequently passed to a feed-forward network where the objects now get the class. The strength of DETR is its end-to-end architecture and it can detect objects reliably. But it also has some major issues, one major limitation is its difficulty in detecting small objects. Furthermore it needs a lot of trainings data and it has a slow convergence. These are reasons why it is not effectively deployed in autonomous driving. However, reliable detection of small objects is crucial, that could be a pedestrian or an important traffic sign. This model forms the foundation for further models like PETR or BEVFormer, which take these problems and try to solve them. Detecting multiple objects using a single camera is essential for autonomous driving. The reason for that is, that a camera is pretty cheap. That's the reason why a lot of researchers try to improve those models. But even there is a huge problem, which is also one huge issue of DETR. For autonomous driving applications, 2D object detection alone is insufficient. Purely 2D detection results in the loss of critical spatial information. In the nuScenes dataset this issue get mentioned and it says that precise 3D bounding boxes are necessary to take reliable decision in autonomous driving. In those applications information about the position in metric distance, yaw-angle and object velocity are crucial [6]. To meet these requirements, transformer-based 3D detection models have been developed. One central approach is PETR, which is the first approach to show that a complete 3D-representation from a camera without a LiDAR is possible [7]. The PETR paper is based on an end-to-end architecture and was developed to address a central weakness of earlier transformer vision models. The problem was, that the previous models could only classify

2D bounding boxes. The PETR model can solve this problem with a transformer architecture to detect 3D object. The paper shows relevance for the autonomous driving systems, because this builds a foundation for 3D object detection with transformer. The model gets multiple input pictures from different angles, which are afterwards put into a CNN Backbone, to extract the feature map from the 2D camera image. A 3D coordinate generator partitions the 3D space into a regular mesh grid in parallel as to see in Fig. 3. Due to the known camera parameters it is possible to transform the 2D-pixel coordinates into a mixed global 3D-coordinate system. Afterwards the collected data is going into a 3D Position Encoder where they build a 3D position-aware-feature. That is the combination between the 2D-Feature and the projected 3D-coordinates. This enables the incorporation of spatial depth information. After that step there are Queries, which then predict the class and the 3D bounding box. The PETR model outperform in the Training on the nuScenes dataset earlier camera only 3D detections like FCOS3D [8]. But also this model has not only advantages, it shows some problems, one of the main limitations lies in the feature computation, which is processed in the image space. That leads to perspective distortions, which shows especially by objects which are further away and they are detected with errors or not detected. Furthermore the model needs to have precise camera calibration, every small issue in the parameter leads to errors in the 3D position.

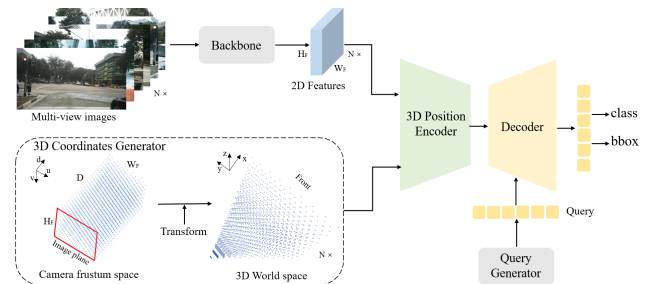


Fig. 3: Described PETR architecture with the multiple image input at the top and the feature extraction and also the parallel 3D coordinate generator at the bottom, which is then connected with a transformer to get a 3D position-aware-feature. Reproduced from [7].

At almost the same time as PETR was presented in mid 2022 there were another approach, the BEVFormer [9]. This architecture explicitly constructs a bird's-eye-view (BEV) representation. With this BEV it gets an even better and robust 3D-detection. Due to the projection of all camera views in a consistent, driving oriented coordinate system the perspective distortion is getting reduced and all object retain their metric structure. This is a huge advantage over PETR, which just operates in the image space. The BEVFormer uses the same query architecture, which was introduced in the DETR Paper [5]. Furthermore they also introduced two attention mechanism as an extension. The so called BEV-Queries are the core of the architecture, they cover the ground around the vehicle as a continuous grid. These Queries then

represent the 3D space around the car and with spatial-cross-Attention they get the relevant image feature from the camera perspective. As a result, a 3D reconstruction is generated in the latent BEV-space, without calculate explicit geometry or depth-maps. Furthermore they also introduce a temporal self-attention in the BEVFormer, this connects the BEV-feature over multiple timesteps. With this mechanism the car can capture model movement and dynamic scenes. This is especially in high dynamic traffic situation useful. The combination between spatial and time attention leads to the fact, that BEVFormer are outperforming other camera methods as PETR, when it comes to detect object which are far away or complex scenes. However the model is more precise and has a better robustly then PETR models, but even it has multiple restrictions, which are essential for autonomous driving. The first problem is the architecture, because of the spatio-temporal attention, this mechanism is computationally expensive, which is then to see in the inference speed. This is to see in Table I, here it shows that the BEVFormer only performs on three FPS. That is far away from the real-time requirements. Recents work on automotive vision emphasizes the need is above 20 FPS to maintain a safe perception latency [10]. The second issue is that the generated BEV-representation need a high model capacity and an extensive GPU-storage, which is difficult for a transfer into an autonomous vehicle. A third limitation is that in BEV-projection the deep ambiguity from camera only models still consists, which leads to the fact that LiDAR are more powerful and they are used together. The Table I illustrates the trade-off between detection accuracy and inference speed, measured in frames per second (FPS). It reports two evaluation metrics: mAP (mean Average Precision), which measures the quality of the predicted bounding boxes and NDS (nuScenes Detection Score), a dataset-specific metric that builds upon mAP while additionally incorporating other weighted factors to provide a more comprehensive assessment of detection performance. While BEVFormer are in points like mAP and NDS ahead of PETR, they lose a lot of speed. That leads to a general pattern in the transformer vision based autonomous driving research. The usage of models which have a high spatial consistence on the other hand have a low latency. This is not optimal for an autonomous vehicle, which has to take decisions in a few milliseconds. Furthermore it needs to mentioned that PETR even with a RTX 3090 is not possible to be used in real-time applications in autonomous systems. As a result, neither model is currently suitable for real-time deployment.

TABLE I: Comparison of PETR, PETRv2, BEVFormer, and BEVFormer v2 on the nuScenes validation set.

Model	Backbone	Image Size	mAP $\uparrow$	NDS $\uparrow$	FPS $\uparrow$
PETR (R50)	ResNet-50	1600 $\times$ 900	0.351	0.421	10.7
PETRv2 (R50)	ResNet-50	1600 $\times$ 640	0.398	0.490	–
PETRv2 (Res101)	ResNet-101	1600 $\times$ 640	0.421	0.524	–
BEVFormer	ResNet-101	1600 $\times$ 900	0.416	0.517	1.7
BEVFormer v2 (B)	InternImage-B	640 $\times$ 1600	0.540	0.620	–
BEVFormer v2 (XL)	InternImage-XL	640 $\times$ 1600	0.556	0.634	–

Development of perception models

The development of camera-only models has continued for clear reason after PETR and BEVFormer demonstrated the potential of transformer-based perception. Therefore new architectures focused on the limitations from previous models. With this there was a new extension of PETR, the PETRv2 [11] which was introduced in the year 2023 and introduces an improved positional embedding transformation which is used to improve the detection of objects at larger distances. The table I shows that the PETRv2 gets better results than the previous model. This clearly illustrates the architectural progression. Also BEVFormer v2 [12] further refined the BEV representation by using a more powerful Intern-Image backbone and refined the spatio-temporal fusion strategy. This results in increased mAP and NDS scores as shown in Table I. However similar to the previous version the model still lacks real-time capability. Recent approaches as StreamPETR [13] optimize the real-time performance through a continuous feature stream, while models such as BEVFusion [14] shows that by combining camera, LiDAR or motion-based transformer the overall system becomes more robust. These developments reveal a clear trend. Efficiency, temporal consistency and robustness of BEV-representation have become the primary design objectives. The foundation for transformer-based 3D perception has been established, but at the moment camera-only models remain insufficient. This leads to the fact, that in the Perception part the transformer-based sensor fusion plays a central role, with multiple sensors the environment can be better described and the objects get detected more accurately. This works very well with the transformer structure [15]. For autonomous driving applications the object detection alone is not enough. A car does not only need to detect the object, it also needs to know where it could be in the next second. This is precisely where transformer-based motion prediction models become relevant, they can model the time dependencies and interactions between multiple road users.

B. Transformer in Prediction

Following the perception stack, the prediction stage is executed in an autonomous driving system. In this stage, the vehicle takes the detected objects and predicts their future positions over subsequent time steps. Classical approaches such as LSTM-based models or feed-forward dynamic models reach their limitations in this setting, as they have limited capacity to modeling long-term temporal dependencies. In addition, they struggle to capture complex multi-agent interactions. An early industrial breakthrough in the field of multimodal trajectory prediction was MultiPath [16]. This work introduced the now widely used anchor-based prediction paradigm. In this approach, fixed trajectory anchors are combined with a Gaussian Mixture Model (GMM), which enables the representation of multiple possible future trajectories along with their associated uncertainties. This idea formed the basis for the successor model MultiPath++ [17]. While MultiPath++ builds upon the original concept and retains the GMM output formulation, it replaces the

static anchors with trainable anchor embeddings in the latent space. These embeddings act as queries within a DETR-like cross-attention decoder, which associates the anchors with features extracted from the scene. Although MultiPath++ incorporates attention-related components such as multi-context gating to fuse road and agent information, it does not utilize a full transformer encoder architecture. As a result, it cannot model global dependencies between all agents and map elements in the scene. This limitation in global scene context modeling motivated the development of modern, fully transformer-based approaches such as Wayformer [18]. Wayformer adopts the decoder structure of MultiPath++ but replaces the entire encoder with a global transformer encoder. This enables the model to capture interactions, temporal dependencies, and multimodal information across the complete scene context, which constitutes a significant advantage in highly dynamic and complex traffic situations. Wayformer employs a relatively simple architecture consisting of a self-attention-based scene encoder that connects one or multiple modalities across temporal and spatial dimensions, as well as a cross-attention decoder, similar to MultiPath++, which models feature dependencies within the scene and predicts future trajectories.

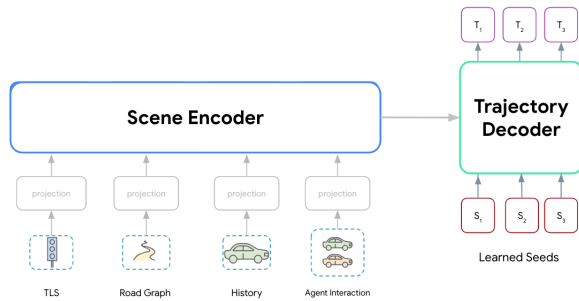


Fig. 4: This figure shows the Wayformer architecture. It shows the different modalities with the Scene Encoder and also the Trajectory Decoder with the learnable seeds. Reproduced from [18].

Figure 4 illustrates the complete architecture of the Wayformer model. In the trajectory decoder, learnable initial queries are introduced, which generate future trajectories through cross-attention. The figure also shows the different modalities processed by the model. Wayformer considers agent history, agent interactions, road graph information, and traffic light signals, all of which are integrated into the model. The model is trained to predict the future trajectory for each agent individually. To enable this, the entire scene is transformed into an egocentric coordinate system, allowing relative position changes to be computed. Before further processing, the projections of the input modalities are scaled to a uniform length. This is achieved through the application of a ReLU activation, after which the dimensions are combined along the temporal and spatial axes. In addition, Wayformer incorporates learnable positional embeddings for each modality to effectively capture their dependencies. After these steps, the scene encoder combines all modalities

and constructs a unified representation of the environment. However, a challenge arises from the fact that the different modalities have heterogeneous feature representations, for example pixel-based features versus object-based features. As a result, some modalities require more computational resources than others. To address this issue, Wayformer introduces three different fusion strategies: late fusion, early fusion, and hierarchical fusion. In late fusion, each modality is processed by its own encoder. All encoders use a uniform embedding dimension and the same number of layers, and the modalities are fused only in the decoder. In contrast, early fusion follows the opposite approach. Instead of assigning a separate encoder to each modality, the modality-specific parameters are reduced at the projection stage. A single self-attention encoder, referred to as the cross-modal encoder, processes all modalities jointly. This approach provides high flexibility while introducing only a small inductive bias. The hierarchical fusion strategy represents a compromise between early and late fusion. In this approach, the model capacity is distributed between a self-attention encoder and a cross-modal encoder, combining elements of both fusion strategies. The Wayformer approach highlights the progression from earlier architectures such as MultiPath++. Instead of relying on a partially transformer-inspired structure, Wayformer employs a fully transformer-based architecture. In addition, the inclusion of different fusion strategies represents a significant improvement over previous models. These design choices result in clear performance gains compared to earlier approaches. The qualitative comparison shown in Figures 5a and 5b provides valuable insights into the differences between the two models. The comparison illustrates that fully transformer-based architectures can outperform models that previously represented the state-of-the-art in trajectory prediction. In the scenario depicted in Figure 5a, Wayformer produces coherent and plausible future trajectories for multiple agents. In contrast, Figure 5b shows that while MultiPath++ generates some valid predictions, several trajectories deviate into implausible, non-drivable regions that do not correspond to drivable areas. This qualitative difference highlights the advantages of employing transformer-based architectures for motion prediction in complex traffic scenarios.

Although Wayformer achieved state-of-the-art performance and significantly outperformed previous models at the time of its publication, it still exhibits several limitations. The egocentric representation must be recomputed for every scene, resulting in high memory consumption and prevents real-time deployment. In addition, the environmental representation remains relatively simplistic, which may cause important aspects to be overlooked or not captured at all. Furthermore, Wayformer predicts the future trajectory for each agent independently. While this design choice is suitable for model evaluation, it is not ideal for solving complex multi-agent interaction tasks. Nevertheless, Wayformer clearly demonstrates the effectiveness of transformer-based architectures for trajectory prediction in autonomous driving. However, it remains far from being deployed in real-

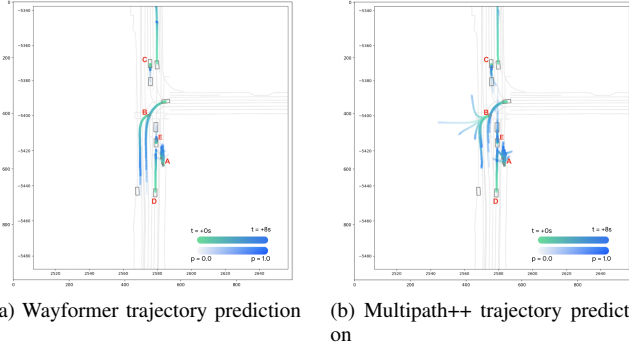


Fig. 5: Comparison of trajectory predictions generated by Wayformer (left) and Multipath++ (right) in a multi-agent traffic scenario. Reproduced from [18].

world systems. These limitations motivated further research, leading to the development of models that address limitations such as multi-agent prediction and richer environment representations. One such model is MTR++ [19], which builds upon its predecessor MTR [20], published around the same time as Wayformer. MTR++ introduces so-called intention queries, which enable the prediction of trajectories across multiple motion modes. This significantly improves multimodal motion prediction for multiple agents, a limitation observed in Wayformer. In addition, MTR++ improves inference latency compared to MTR. While MTR++ achieves strong performance in predicting future positions, it still exhibits several challenges. One limitation is its relatively low accuracy, with an mAP of approximately, although it achieves a favorable miss rate of 0.4106. Another issue is that the multiple predicted future trajectories are often very similar, making the agent’s underlying intention difficult to infer. Further research is therefore required to improve both trajectory diversity and prediction accuracy in order to enable the deployment of transformer-based prediction models in real-time autonomous driving systems. Nonetheless, the rapid evolution of transformer architectures in this domain demonstrates their strong potential for motion prediction tasks.

TABLE II: Comparison of transformer-based motion prediction models on the Waymo Open Motion Dataset (WOMD).

Method	Task	minADE ↓	minFDE ↓	mAP ↑
Wayformer	Marginal	0.545	1.126	0.412
MTR	Joint	0.9132	2.0536	0.1992
MTR++	Joint	0.8859	1.9712	0.2398
MotionLM	Joint	0.8911	2.0067	0.2178

Note: Marginal and joint prediction correspond to different task definitions and are not directly comparable.

Table II compares the Wayformer, MTR, and MTR++ architectures. The reported metrics include minADE (minimum Average Displacement Error), which measures the average deviation between the predicted and ground-truth trajectories, minFDE (minimum Final Displacement Error), which measures the deviation at the final prediction time

step, and mAP, as described previously. Not all architectures can be directly compared, as Wayformer was evaluated only on the marginal motion prediction task. Marginal prediction considers each traffic participant independently and does not explicitly model interactions between agents. While this setting is not fully representative of real-world autonomous driving scenarios, Wayformer is included in the table to provide a complete illustration of the progression of transformer-based prediction models. In contrast, both MTR and MTR++ are evaluated on joint motion prediction tasks using the interactive validation set. This setting is more relevant for autonomous driving, as it jointly models multiple agents within the same traffic scene. Under this formulation, the reported metrics show a clear performance improvement from MTR to MTR++, highlighting the effectiveness of the architectural extensions introduced in MTR++.

In addition to MTR++, another recent development in trajectory prediction is MotionLM [21]. Unlike the MTR++ architecture, MotionLM treats motion as a language modeling problem and represents trajectories as discrete tokens. As a result, it does not rely on anchor-based formulations or explicit latent variables to model multimodal distributions. Instead, MotionLM generates a distribution over future agent interactions within a single autoregressive decoding process. This approach achieved state-of-the-art performance on the Waymo Open Dataset in the year of its publication. The quantitative results for joint motion prediction are reported in Table II. While MotionLM shows improvements compared to MTR, it does not outperform MTR++. However, given its fundamentally different modeling paradigm, these results are notable and highlight the potential of language-inspired approaches for motion prediction. Consequently, MotionLM is particularly interesting for future research, as it introduces a novel perspective on trajectory prediction. Despite its promising performance, MotionLM also exhibits several limitations. One key issue is error accumulation, as predictions are generated autoregressively based on previous outputs. Consequently, small prediction errors can propagate over time, causing the predicted trajectory to drift increasingly away from the ground truth. In addition, MotionLM struggles in scenarios where the scene is not fully observed, leading to missing contextual information and, in turn, inaccurate predictions. Overall, the discussed models demonstrate that transformer architectures play a crucial role in the prediction stack of autonomous driving systems. The presented approaches reveal a clear research trend toward transformer-based models, to the extent that trajectories are now modeled as a form of language. These models lay the foundation for modern research in motion prediction. However, it is also evident that transformer-based prediction models are not yet fully suitable for deployment in real-world autonomous driving systems. The underlying reasons for these limitations are discussed in more detail in the following section.

IV. LIMITATIONS AND FUTURE DIRECTIONS

Transformer architectures enable novel approaches for autonomous driving through the use of self-attention mecha-

nisms. They have significantly advanced both the perception and prediction stacks and outperform conventional methods in many scenarios. Despite these substantial advantages, transformer-based approaches also exhibit notable limitations. At present, there is no published work that demonstrates the deployment of transformer architectures in real-world autonomous driving systems. Instead, all reported state-of-the-art results are obtained exclusively on benchmark datasets rather than in real driving environments. This indicates that transformer architectures still face several unresolved challenges. One fundamental limitation of transformer models is their dependence on large amounts of training data in order to generalize effectively to new scenarios. Due to their architectural complexity, extensive data is required to train the attention mechanisms. However, such data is expensive and limited in the context of autonomous driving, which makes the deployment of transformer models less practical. In addition, latency and real-time capability remain major concerns. Although transformer models achieve strong performance and can capture global context, they lack the real-time efficiency required for safety-critical applications such as autonomous driving. This issue is also reflected in Table I, where inference speeds remain well below 30 FPS, even when using high-end GPUs. For safety-critical scenarios, substantially higher frame rates are often required. This limitation is closely related to current hardware constraints. While the industry is rapidly advancing specialized AI hardware, existing solutions are still insufficient for deployment in autonomous vehicles, where low power consumption, limited memory capacity, and high throughput are essential. Although new AI accelerators are under active development, current hardware platforms do not yet meet all of these requirements. Another challenge is the limited interpretability of transformer architectures. These models are often considered black-box algorithms, which poses a significant problem for safety-critical applications. In autonomous driving, it is crucial to understand how and why specific decisions are made at each processing step, which is difficult to achieve with highly complex and opaque models. Furthermore, many transformer-based perception architectures still rely on CNN backbones. This often requires image resizing, increasing computational cost and potentially degrading the global contextual representation of the scene. While replacing CNN backbones with vision transformers could mitigate this issue, vision transformers typically require substantially larger training datasets, making this approach currently impractical. Beyond these limitations, recent studies show that transformer models struggle in rare and critical scenarios that are underrepresented during training. This behavior can be attributed to the long-tailed distribution present in commonly used autonomous driving datasets. As shown by [22], datasets such as nuScenes and AV2 exhibit significant class imbalance, causing safety-critical scenarios to be insufficiently learned despite the large overall dataset size. Addressing these challenges remains an important goal for future research. The transformation undergone by transformer models is remarkable, as they have

evolved from an architecture originally designed for natural language processing into a central paradigm in autonomous driving research. A key objective of ongoing work is the development of end-to-end transformer-based systems, which could potentially reduce error propagation and further improve model accuracy.

V. CONCLUSIONS

This review paper presented the development of transformer models in autonomous driving applications. A clear trend emerges: transformer architectures represent a major advancement for the autonomous driving domain. In particular, transformer-based models achieve state-of-the-art performance in the perception and prediction stacks, where they outperform classical approaches such as RNN- and CNN-based methods in many tasks. The results discussed in this review demonstrate that, through self-attention mechanisms, transformer models are capable of capturing global context, which enables strong performance in multimodal and complex traffic scenarios. Despite these advances, several open challenges remain and require further investigation. At present, the deployment of transformer models in real-world autonomous vehicles is still limited, primarily due to constraints related to real-time capability. Nevertheless, the foundations for future progress have been established. While transformer models already demonstrate superior performance, further optimization in terms of interpretability, computational efficiency, and latency is necessary to enable their reliable use in autonomous driving systems. Overall, the direction of research is clear. Transformer architectures have the potential to significantly advance autonomous driving. Although they are not yet ready for large-scale real-world deployment, continued research and development are likely to further mature these models and position them as a central component of future autonomous driving technologies.

REFERENCES

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," 2017. [Online]. Available: <https://arxiv.org/abs/1706.03762>
- [2] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," 2021. [Online]. Available: <https://arxiv.org/abs/2010.11929>
- [3] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," 2019. [Online]. Available: <https://arxiv.org/abs/1810.04805>
- [4] J. Ngiam, B. Caine, V. Vasudevan, Z. Zhang, H.-T. L. Chiang, J. Ling, R. Roelofs, A. Bewley, C. Liu, A. Venugopal, D. Weiss, B. Sapp, Z. Chen, and J. Shlens, "Scene transformer: A unified architecture for predicting multiple agent trajectories," 2022. [Online]. Available: <https://arxiv.org/abs/2106.08417>
- [5] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-end object detection with transformers," 2020. [Online]. Available: <https://arxiv.org/abs/2005.12872>
- [6] H. Caesar, V. Bankiti, A. H. Lang, S. Vora, V. E. Liong, Q. Xu, A. Krishnan, Y. Pan, G. Baldan, and O. Beijbom, "nuscenes: A multimodal dataset for autonomous driving," 2020. [Online]. Available: <https://arxiv.org/abs/1903.11027>
- [7] Y. Liu, T. Wang, X. Zhang, and J. Sun, "Petr: Position embedding transformation for multi-view 3d object detection," 2022. [Online]. Available: <https://arxiv.org/abs/2203.05625>

- [8] T. Wang, X. Zhu, J. Pang, and D. Lin, "Fc3d: Fully convolutional one-stage monocular 3d object detection," 2021. [Online]. Available: <https://arxiv.org/abs/2104.10956>
- [9] Z. Li, W. Wang, H. Li, E. Xie, C. Sima, T. Lu, Q. Yu, and J. Dai, "Bevformer: Learning bird's-eye-view representation from multi-camera images via spatiotemporal transformers," 2022. [Online]. Available: <https://arxiv.org/abs/2203.17270>
- [10] D. Gehrig and D. Scaramuzza, "Low-latency automotive vision with event cameras," *Nature*, vol. 629, no. 8014, pp. 1034–1040, 2024.
- [11] Y. Liu, J. Yan, F. Jia, S. Li, A. Gao, T. Wang, X. Zhang, and J. Sun, "PetrV2: A unified framework for 3d perception from multi-camera images," 2022. [Online]. Available: <https://arxiv.org/abs/2206.01256>
- [12] C. Yang, Y. Chen, H. Tian, C. Tao, X. Zhu, Z. Zhang, G. Huang, H. Li, Y. Qiao, L. Lu, J. Zhou, and J. Dai, "Bevformer v2: Adapting modern image backbones to bird's-eye-view recognition via perspective supervision," 2022. [Online]. Available: <https://arxiv.org/abs/2211.10439>
- [13] S. Wang, Y. Liu, T. Wang, Y. Li, and X. Zhang, "Exploring object-centric temporal modeling for efficient multi-view 3d object detection," 2023. [Online]. Available: <https://arxiv.org/abs/2303.11926>
- [14] Z. Liu, H. Tang, A. Amini, X. Yang, H. Mao, D. Rus, and S. Han, "Bevfusion: Multi-task multi-sensor fusion with unified bird's-eye view representation," 2024. [Online]. Available: <https://arxiv.org/abs/2205.13542>
- [15] C. Zhou, L. Yu, A. Babu, K. Tirumala, M. Yasunaga, L. Shamsi, J. Kahn, X. Ma, L. Zettlemoyer, and O. Levy, "Transfusion: Predict the next token and diffuse images with one multi-modal model," 2024. [Online]. Available: <https://arxiv.org/abs/2408.11039>
- [16] Y. Chai, B. Sapp, M. Bansal, and D. Anguelov, "Multipath: Multiple probabilistic anchor trajectory hypotheses for behavior prediction," 2019. [Online]. Available: <https://arxiv.org/abs/1910.05449>
- [17] B. Varadarajan, A. Hefny, A. Srivastava, K. S. Refaat, N. Nayakanti, A. Cornman, K. Chen, B. Douillard, C. P. Lam, D. Anguelov, and B. Sapp, "Multipath++: Efficient information fusion and trajectory aggregation for behavior prediction," 2021. [Online]. Available: <https://arxiv.org/abs/2111.14973>
- [18] N. Nayakanti, R. Al-Rfou, A. Zhou, K. Goel, K. S. Refaat, and B. Sapp, "Wayformer: Motion forecasting via simple and efficient attention networks," 2022. [Online]. Available: <https://arxiv.org/abs/2207.05844>
- [19] S. Shi, L. Jiang, D. Dai, and B. Schiele, "Mtr++: Multi-agent motion prediction with symmetric scene modeling and guided intention querying," 2024. [Online]. Available: <https://arxiv.org/abs/2306.17770>
- [20] —, "Motion transformer with global intention localization and local movement refinement," 2023. [Online]. Available: <https://arxiv.org/abs/2209.13508>
- [21] A. Seff, B. Cera, D. Chen, M. Ng, A. Zhou, N. Nayakanti, K. S. Refaat, R. Al-Rfou, and B. Sapp, "Motionlm: Multi-agent motion forecasting as language modeling," 2023. [Online]. Available: <https://arxiv.org/abs/2309.16534>
- [22] N. Peri, A. Dave, D. Ramanan, and S. Kong, "Towards long-tailed 3d detection," 2023. [Online]. Available: <https://arxiv.org/abs/2211.08691>